

```

//
// Funzioni per soluzioni di ODE con metodi iterativi
//
// ode_LotkaVolterraParams ode_g_lotka_volterra{1.5, 1.0, 3.0, 1.0}; // aggressivi
ode_LotkaVolterraParams ode_g_lotka_volterra{1.0, 1.0, 1.0, 1.0}; // soft per collaudo

void ode_set_lotka_volterra_params(
    double alpha,
    double beta,
    double gamma,
    double delta)
{
    ode_g_lotka_volterra.alpha = alpha;
    ode_g_lotka_volterra.beta = beta;
    ode_g_lotka_volterra.gamma = gamma;
    ode_g_lotka_volterra.delta = delta;
}

void ode_get_lotka_volterra_params(
    double& alpha,
    double& beta,
    double& gamma,
    double& delta)
{
    alpha = ode_g_lotka_volterra.alpha;
    beta = ode_g_lotka_volterra.beta;
    gamma = ode_g_lotka_volterra.gamma;
    delta = ode_g_lotka_volterra.delta;
}

// Oscillatore semplice e smorzato (vector dim 2)
double ode_oscillator_omega = 1.0;
double ode_oscillator_gamma = 0.2;

void ode_set_oscillator_omega( // helper/setter per cambiare frequenza all' osc armonico
    double omega)
{
    ode_oscillator_omega = omega;
}

void ode_set_oscillator_gamma( // helper/setter per cambiare frequenza all' osc armonico smorzato
    double gamma)
{
    ode_oscillator_gamma = gamma;
}

// Logistic (scalar)
ode_LogisticParams ode_g_logistic {1.0, 10.0, 1.0};

void ode_set_logistic_params(
    double r,
    double k,
    double y0)
{
    ode_g_logistic.r = r;
    ode_g_logistic.K = k;
    ode_g_logistic.y0 = y0;
}

```

// calcoli dei valori esatti per stima dell' errore

```
double ode_scalar_exact_logistic(double t)
{
    double r = ode_g_logistic.r;
    double K = ode_g_logistic.K;
    double y0 = ode_g_logistic.y0;

    return (y0 * K * std::exp(r * t)) /
        ((K - y0) + y0 * std::exp(r * t));
}

double ode_vec2_exact_damped_q(
    double t,
    double q0,
    double p0)
{
    const double omega = ode_oscillator_omega;
    const double gamma = ode_oscillator_gamma;
    const double Omega = std::sqrt(omega * omega - gamma * gamma);

    const double A = q0;
    const double B = (p0 + gamma * q0) / Omega;

    return std::exp(-gamma * t) *
        (A * std::cos(Omega * t) + B * std::sin(Omega * t));
}

double ode_vec2_exact_damped_p(
    double t,
    double q0,
    double p0)
{
    const double omega = ode_oscillator_omega;
    const double gamma = ode_oscillator_gamma;
    const double Omega = std::sqrt(omega * omega - gamma * gamma);

    const double A = q0;
    const double B = (p0 + gamma * q0) / Omega;

    const double et = std::exp(-gamma * t);
    const double c = std::cos(Omega * t);
    const double s = std::sin(Omega * t);

    return et * ((-gamma * A + Omega * B) * c
        + (-gamma * B - Omega * A) * s);
}
```

```

double ode_vec2_exact_oscillator_q(
    double t,
    double q0,
    double p0)
{
    const double omega = ode_oscillator_omega;
    const double wt = omega * t;

    return q0 * std::cos(wt) + (p0 / omega) * std::sin(wt);
}

double ode_vec2_exact_oscillator_p(
    double t,
    double q0,
    double p0)
{
    const double omega = ode_oscillator_omega;
    const double wt = omega * t;

    return -omega * q0 * std::sin(wt) + p0 * std::cos(wt);
}

// RHS Scalari (exp decay e logistica)

// scalare:  $y' = \lambda * y$ 
double ode_scalar_rhs_decay(
    double t,
    double y)
{
    (void)t;
    const double lambda = -1.0;
    return lambda * y;
}

// logistica  $y' = r/k * y (k - y)$ 
double ode_scalar_rhs_logistic(
    double t,
    double y)
{
    (void)t;
    return ode_g_logistic.r * y * (1.0 - y / ode_g_logistic.K);
}

```

```

// Stepper scalari (eulero, heun, runge-kutta 4th)

// Eulero
void ode_scalar_step_euler(
    double t,
    double h,
    double* y,
    double (*f)(double, double))
{
    *y = *y + h * f(t, *y);
}

// Heun (Eulero migliorato, ordine 2)
void ode_scalar_step_heun(
    double t,
    double h,
    double* y,
    double (*f)(double, double))
{
    const double k1 = f(t, *y);
    const double k2 = f(t + h, *y + h * k1);
    *y = *y + 0.5 * h * (k1 + k2);
}

// Runge-Kutta 4th (ordine 4)
void ode_scalar_step_rk4(
    double t,
    double h,
    double* y,
    double (*rhs)(double, double))
{
    double k1 = rhs(t, *y);
    double k2 = rhs(t + 0.5 * h, *y + 0.5 * h * k1);
    double k3 = rhs(t + 0.5 * h, *y + 0.5 * h * k2);
    double k4 = rhs(t + h, *y + h * k3);

    *y = *y + (h / 6.0) * (k1 + 2.0 * k2 + 2.0 * k3 + k4);
}

```

```

// RHS vettoriale 2D (predatore-preda e oscillatore)

// Oscillatore smorzato
void ode_vec2_rhs_damped(
    double t,
    const double* y,
    double* dydt,
    int n)
{
    (void)t;
    (void)n; // assumiamo n == 2

    const double omega = ode_oscillator_omega;
    const double gamma = ode_oscillator_gamma;

    dydt[0] = y[1]; // q' = p
    dydt[1] = -2.0 * gamma * y[1] - omega * omega * y[0]; // p' = -2 gamma p - omega^2 q
}

// predatore-preda
void ode_vec2_rhs_lotkavolterra(
    double t,
    const double* y,
    double* dydt,
    int n)
//
// x' = x - xy
// y' = -y + xy
//
{
    (void)t;
    (void)n; // assumiamo n == 2

    const double alpha = ode_g_lotka_volterra.alpha; // crescita prede
    const double beta = ode_g_lotka_volterra.beta; // predazione
    const double gamma = ode_g_lotka_volterra.gamma; // mortalità predatori
    const double delta = ode_g_lotka_volterra.delta; // crescita predatori per incontro

    const double x = y[0]; // prede
    const double z = y[1]; // predatori

    dydt[0] = alpha * x - beta * x * z;
    dydt[1] = delta * x * z - gamma * z;
}

```

```

// Oscillatore armonico libero
void ode_vec2_rhs_oscillator(
    double t,
    const double* y,
    double* dydt,
    int n)
{
    (void)t;
    (void)n; // assumiamo n == 2
    const double omega = ode_oscillator_omega;

    // const double a11 = 0.0, a12 = 1.0;
    // const double a21 = -1.0, a22 = 0.0;
    // dydt[0] = a11 * y[0] + a12 * y[1];
    // dydt[1] = a21 * y[0] + a22 * y[1];
    dydt[0] = y[1]; // q' = p
    dydt[1] = -omega * omega * y[0]; // p' = -omega^2 q
}

```

```

// Stepper vettoriali dim N generica

// Eulero dim N
void ode_vecN_step_euler(
    double t,
    double h,
    double* y,           // equivalente a Vec& y
    int n,               // equivalente a Vec& y se n == y.size()
    void (*rhs)(         // puoi chiamarla anche nando, e' la f a secondo membro
        double,
        const double*,
        double*,
        int)
    )
{
    std::vector<double> dydt(n);
    rhs(t, y, dydt.data(), n);
    for (int i = 0; i < n; ++i) {
        y[i] += h * dydt[i];
    }
}

void ode_vecN_step_heun(
    double t,
    double h,
    double* y,
    int n,
    void (*rhs)(
        double,
        const double*,
        double*,
        int)
    )
{
    std::vector<double> k1(n), k2(n), ytmp(n);

    rhs(t, y, k1.data(), n);           // k1

    for (int i = 0; i < n; ++i)
        ytmp[i] = y[i] + h * k1[i];    // predictor

    rhs(t + h, ytmp.data(), k2.data(), n); // k2

    for (int i = 0; i < n; ++i)
        y[i] += 0.5 * h * (k1[i] + k2[i]); // corrector
}

```

```

void ode_vecN_step_rk2(
    double t,
    double h,
    double* y,
    int n,
    void (*rhs)(
        double,
        const double*,
        double*,
        int)
    )
{
    std::vector<double> k1(n), k2(n), ytmp(n);

    rhs(t, y, k1.data(), n);

    for (int i = 0; i < n; ++i)
        ytmp[i] = y[i] + 0.5 * h * k1[i];

    rhs(t + 0.5 * h, ytmp.data(), k2.data(), n);

    for (int i = 0; i < n; ++i)
        y[i] += h * k2[i];
}

void ode_vecN_step_rk4(
    double t,
    double h,
    double* y,
    int n,
    void (*rhs)(
        double,
        const double*,
        double*,
        int)
    )
{
    std::vector<double> k1(n), k2(n), k3(n), k4(n), ytmp(n);

    rhs(t, y, k1.data(), n);

    for (int i = 0; i < n; ++i)
        ytmp[i] = y[i] + 0.5 * h * k1[i];
    rhs(t + 0.5 * h, ytmp.data(), k2.data(), n);

    for (int i = 0; i < n; ++i)
        ytmp[i] = y[i] + 0.5 * h * k2[i];
    rhs(t + 0.5 * h, ytmp.data(), k3.data(), n);

    for (int i = 0; i < n; ++i)
        ytmp[i] = y[i] + h * k3[i];
    rhs(t + h, ytmp.data(), k4.data(), n);

    for (int i = 0; i < n; ++i)
        y[i] += (h / 6.0) * (k1[i] + 2.0 * k2[i] + 2.0 * k3[i] + k4[i]);
}

```



```

// test sul decadimento scalare
int ode_scalar_test_euler_decay()
{
    const double t0 = 0.0;
    const double T  = 5.0;
    const double h  = 0.1;
    const int     N  = static_cast<int>((T - t0) / h);

    double t = t0;
    double y = 1.0;

    for (int n = 0; n <= N; ++n) {
        double y_exact = std::exp(-1.0 * (t - t0));
        std::cout << t << " " << y << " " << y_exact << "\n";

        if (n < N) {
            ode_scalar_step_euler(t, h, &y, ode_scalar_rhs_decay);
            t += h;
        }
    }
    return 0;
}

int ode_scalar_test_euler_logistic()
{
    const double t0 = 0.0;
    const double T  = 5.0;
    const double h  = 0.1;
    const int     N  = static_cast<int>((T - t0) / h);

    double t = t0;
    double y = 1.0;

    for (int n = 0; n <= N; ++n) {
        double y_exact = std::exp(-1.0 * (t - t0));
        std::cout << t << " " << y << " " << y_exact << "\n";

        if (n < N) {
            ode_scalar_step_euler(t, h, &y, ode_scalar_rhs_logistic);
            t += h;
        }
    }
    return 0;
}

```

```

int ode_scalar_test_heun_decay()
{
    const double t0 = 0.0;
    const double T  = 5.0;
    const double h  = 0.1;
    const int     N  = static_cast<int>((T - t0) / h);

    double t = t0;
    double y = 1.0; // y0

    for (int n = 0; n <= N; ++n) {
        // soluzione esatta sul nodo (per confronto)
        double y_exact = std::exp(-1.0 * (t - t0));
        std::cout << t << " " << y << " " << y_exact << "\n";

        if (n < N) {
            ode_scalar_step_heun(t, h, &y, ode_scalar_rhs_decay);
            t += h;
        }
    }

    return 0;
}

```

```

int ode_scalar_test_heun_logistic()
{
    const double t0 = 0.0;
    const double T = 5.0;
    const double h = 0.1;
    const int N = static_cast<int>((T - t0) / h);

    double t = t0;
    double y = 1.0; // y0

    for (int n = 0; n <= N; ++n) {
        // soluzione esatta sul nodo (per confronto)
        double y_exact = std::exp(-1.0 * (t - t0));
        std::cout << t << " " << y << " " << y_exact << "\n";

        if (n < N) {
            ode_scalar_step_heun(t, h, &y, ode_scalar_rhs_logistic);
            t += h;
        }
    }

    return 0;
}

int ode_scalar_test_rungekutta4_decay()
{
    const double t0 = 0.0;
    const double T = 5.0;
    const double h = 0.1;
    const int N = static_cast<int>((T - t0) / h);

    double t = t0;
    double y = 1.0; // y0

    for (int n = 0; n <= N; ++n) {
        // soluzione esatta sul nodo (per confronto)
        double y_exact = std::exp(-1.0 * (t - t0));
        std::cout << t << " " << y << " " << y_exact << "\n";

        if (n < N) {
            ode_scalar_step_rk4(t, h, &y, ode_scalar_rhs_decay);
            t += h;
        }
    }

    return 0;
}

```